Evaluating Language Models for Austrian Tax Law Question Answering: GPT-2, QLoRA Fine-Tuning, and RAG

By Anna Sophie Windisch

Course: 4805 Applications of Data Science

Instructor: Dr. Svitlana Vakulenko

Date of Submission: 15. April 2026

Table of Contents

1 Introduction 3

1.1 Task 3

1.2 Data 3

1.2.1 Test Dataset 3

1.2.2 Fine-tuning dataset 4

1.2.3 RAG Document Collection 5

2 Models 6

2.1 GPT2-large – Inference-Only 6

2.2 Llama 3.1 8B – Supervised Fine-Tuning via QLoRA 7

2.3 Qwen 2.5 3B + multilingual-E5-base – RAG 11

3 Code 16

4 Model Evaluation 17

4.1 Metrics 17

4.2 Results 18

4.3 Error Analysis 19

5 Conclusion 22

6 Sources 23

1 Introduction

1.1 Task

The goal of this project was to use, train and evaluate **three different language models** for the task of question answering on **Austrian tax law**. Given a question about Austrian tax legislation, each system had to generate a natural-language answer, which should ideally correspond with the related paragraph.

All students worked with the same dataset and were asked to submit three different model variations:

1. **Inference only**: Using an existing pre-trained model without any modifications
2. **Supervised fine-tuning**: Adapting a pre-trained model to the domain using custom training data
3. **RAG (Retrieval-Augmented Generation)**: Extending a language model with a retrieval system that searches actual Austrian law texts before generating an answer

The models were evaluated against reference answers from a common dataset created by the students collectively.

1.2 Data

1.2.1 Test Dataset

dataset_clean.csv

The shared test dataset contains **643 questions about Austrian tax law**, each with a unique ID, a question text (prompt), and a reference answer. The dataset was used exclusively as a test set (not for fine-tuning). The dataset was created by the students collectively, each student writing 20 prompts + answer based on an assigned Austrian tax paragraph.

The questions cover the following thematic categories:

Category ID	Topic	Count
VAT-INTL	Value added tax — international	82
SELF	Self-employment income (EStG)	82
GRESt-AT	Real estate transfer tax (GrEStG)	81
TAX-INTL	International tax law	80
VAT-DOM	Value added tax — domestic	80
CORP-TAX	Corporate income tax (KStG)	60
EMP	Employment income (EStG)	60
EStG-23	Business income § 23 EStG	60
ESTG27	Capital income § 27 EStG	38
ESTG-NEW	Recent EStG amendments	20

Example questions include:

- *"Was sind Einkünfte aus Gewerbebetrieb gemäß § 23 EStG?"*
- *"Welche steuerlichen Folgen hat ein zinsloses Darlehen einer Kapitalgesellschaft an ihre Gesellschafter?"*
- *"Unter welchen Voraussetzungen ist ein Grundstückserwerb von der Grunderwerbsteuer befreit?"*

1.2.2 Fine-tuning dataset

austrian_tax_law_training_dataset.json

Since the course test set was not permitted for training, I created a custom training dataset using AI, as there was no publicly available QA dataset on Austrian tax law. The dataset was created using Claude AI (claude-opus-4-20250514), by extracting relevant paragraphs from official law texts and prompting the AI to generate 250 question-answer pairs based on the provided texts.
The source laws used were: EStG 1988, UStG 1994, KStG 1988, and GrEStG 1987, downloaded from the Austrian Legal Information System (https://data.bka.gv.at/ris/api/v2.6/Bundesrecht).

The resulting dataset contains **253 question-answer pairs** in JSON format with the following structure:

```
{
  "id": "ESTG-1-001",
  "prompt": "Wer unterliegt in Österreich der unbeschränkten Einkommensteuerpflicht?",
  "correct_answer": "Der unbeschränkten Einkommensteuerpflicht unterliegen jene natürlichen Personen, die im Inlar
  "sources": [
    "§ 1 Abs. 2 EStG 1988"
  ]
```

1.2.3 RAG Document Collection

alle_chunks.json

For the RAG system, the same four Austrian tax laws were downloaded from RIS: EStG 1988, UStG 1994, KStG 1988, and GrEStG 1987. The texts were then parsed and split into **6.685 text chunks** *(see section 2.3 for preprocessing details)*.

2 Models

2.1 GPT2-large – Inference-Only

For the inference only part I used **GPT2-large**.
I loaded the model from HuggingFace and used it to generate answers for all 643 test questions without any fine-tuning or additional context. Each question was passed directly to the model, which then generated a response with up to 100 new tokens.

Code Reference:

```
# loading the models
# loeading gpt-2 large as a text generator pipeline
generator = pipeline('text-generation', model='gpt2-large', device=device_id)   # device=device_id sends the model to GPU/CPU

# setting the padding token as the end of sequence token
# GPT-2 has no pad token so we use end of sequence to prevent tokenizer warning
generator.tokenizer.pad_token_id = generator.tokenizer.eos_token_id

print("Pipeline loaded.") # confirmation if model loaded successfully
```

```
Loading weights: 100%|██████████████████████████████████████████████████| 436/436 [00:00<00:00, 1809.21it/s]
Pipeline loaded.
```

<u>Model Details:</u>

→ Model: gpt2-large

→ Parameters: 774 million

→ Architecture: Decoder-only transformer (12 layers, 20 attention heads, 1280 hidden dim)

→ Pre-training data: WebText (around 40 GB of English web text, sourced from Reddit outbound links)

<u>Hyperparameters:</u>

max_new_tokens = 100
→ limits the response to a maximum of 100 newly generated tokens

num_return_sequences = 1
→ means the model will generate only one output sequence

truncation = True
→ input exceeding the model's maximum length will be shortened to fit

return_full_text = False
→ ensures that only the generated continuation is returned, without including the original input prompt

Code Reference:

```
# sending the promt to the model and generating a response
output = generator(
    row[COL_PROMPT],          # the question / promt text that is being processed
    max_new_tokens=100,       # generating at most 100 tokens
    num_return_sequences=1,   # returning only 1 answer per promt
    truncation=True,          # truncating (cutting off) end of input if it exceeds maximum model length
    return_full_text=False    # returning only the generated anser, not the origninal prompt again
)
```

<u>Platform used:</u>

Hugging Face pipeline API, run on GPU (CUDA).

2.2 Llama 3.1 8B – Supervised Fine-Tuning via QLoRA

For the fine-tuning part I decided to use **Llama 3.1 8B**, which is a large, instruction-tuned language model.
I fine-tuned this model on my custom Austrian tax law training dataset using **QLoRA**, which is a memory efficient fine-tuning technique that makes it possible to train a 8-billion-paramter model on a single free GPU.
The fine-tuned model was then used to answer the 643 test questions.

Code Reference:

```
from unsloth import FastLanguageModel        # Import Unsloth's optimized model loader

max_seq_length = 2048                                          # Maximum number of tokens the model can process at once (2048 is enough for tax law)

model, tokenizer = FastLanguageModel.from_pretrained(          # Load the pre-trained model and its tokenizer together
    model_name = "unsloth/Meta-Llama-3.1-8B-Instruct-bnb-4bit", # Use Meta's Llama 3.1 8B instruct model, pre-quantized to 4-bit by Unsloth
    max_seq_length = max_seq_length,                           # Pass the max token length defined above
    dtype = None,                                              # Let Unsloth automatically pick the best data type (e.g., float16 or bfloat16)
    load_in_4bit = True,                                       # Load model weights in 4-bit precision (QLoRA) — drastically reduces VRAM usage
)
```

<u>Why using QLoRA was relevant:</u>

The full Llama 3.1 8B model has 8 billion parameters. Training all of them would normally require around 80Gb of GPU memory, which is far more than what is freely available on platforms like Google Colab. This issue can be avoided by QLoRA in two steps: Firstly, the model weights are quantized, meaning the weights are compressed to 4-bit precision (instead of 16-bit or 32-bit). These weights are then frozen and never updated during the training, which reduces the memory usage by around 75%. Secondly, instead of the original weights being updated (as they are frozen), small additional weight matrices called “LoRA adapters” are added to the key attention layers. These adapters are very small, only about 0.5% of the model’s parameters, and they are trained in 16-bit precision. They then learn the domain-specific adjustments needed for Austrian tax law. Consequentially, the model can learn while only updating a small part of its parameters, which allows fine-tuning on a simple T4 GPU with 16 GB of VRAM.

Base Model Details:

→ Model: unsloth/Meta-Llama-3.1-8B-Instruct-bnb-4bit

→ Parameters: 8 billion

→ Architecture: Decoder-only transformer (Llama architecture)

→ Pre-training data: around 15 trillion tokens of multilingual text, code, and curated data (Meta AI, cutoff Sept. 2024)

→ Quantization: 4-bit (BitsAndBytes NF4), pre-quantized by Unsloth

→ Fine-tuning library: Unsloth + TRL SFTTrainer

LoRA Configuration:

LoRA rank (r) = 16
→ Controls how many new parameters are added per adapter layer. Higher = more expressive but more memory. 16 is a standard balanced default.

LoRA alpha = 16
→ Scaling factor for the LoRA updates. Usually set equal to r.

LoRA dropout = 0
→ No dropout on the LoRA layers (recommended by Unsloth for speed and stability).

Target modules = [sq_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj]
→ The specific weight matrices inside the transformer that get LoRA adapters. These are the attention projections and the feed-forward network layers (the most important components for learning new knowledge).

Bias = none
→ Bias parameters are not trained, which keeps fine-tuning minimal.

Gradient check pointing = unsloth
→ Saves VRAM by recomputing intermediate activations using the unsloth gradient checkpoint instead of storing them (at a small speed cost).

Code Reference:

```
model = FastLanguageModel.get_peft_model(              # Wrap the model with LoRA adapters (PEFT = Parameter-Efficient Fine-Tuning)
    model,                                             # base model (loaded in the previous cell)
    r = 16,                                            # LoRA rank: controls how many new parameters are added, 16 is balanced default
    target_modules = [                                 # List of transformer sub-layers to attach LoRA adapters to
        "q_proj",                                      # Query projection matrix in the attention mechanism
        "k_proj",                                      # Key projection matrix in the attention mechanism
        "v_proj",                                      # Value projection matrix in the attention mechanism
        "o_proj",                                      # Output projection after the attention heads
        "gate_proj",                                   # Gate layer inside the feed-forward network (FFN)
        "up_proj",                                     # Up-projection inside the FFN
        "down_proj"                                    # Down-projection inside the FFN
    ],
    lora_alpha = 16,                                   # Scaling factor for the LoRA updates; usually set equal to r
    lora_dropout = 0,                                  # Dropout rate for LoRA layers; 0 = disabled (recommended by Unsloth for speed)
    bias = "none",                                     # Do not train any bias parameters — keeps training minimal
    use_gradient_checkpointing = "unsloth",            # Use Unsloth's gradient checkpointing to save VRAM at a small speed cost
    random_state = 42,                                 # Fix the random seed so results are reproducible
)

# Shows how many parameters are actually being trained
model.print_trainable_parameters()                     # Print how many parameters will actually be updated during training (should be ~0.5% of total)
```

```
Unsloth 2026.4.4 patched 32 layers with 32 QKV layers, 32 O layers and 32 MLP layers.
trainable params: 41,943,040 || all params: 8,072,204,288 || trainable%: 0.5196
```

Training Hyperparameters:

Epochs = 3
→ The model sees the full training dataset 3 times.

Batch size per device = 2
→ 2 training examples are processed simultaneously per GPU step.

Gradient accumulation steps = 4
→ Gradients are accumulated over 4 steps before updating weights. Effective batch size = 2 × 4 = 8. This simulates a larger batch without needing more memory.

Learning rate= 2e-4
→ Step size for weight updates (0.0002), which is a standard for QLoRA.

LR scheduler = linear
→ The learning rate decreases linearly from 2e-4 to 0 over training.

Warmup steps = 5
→ The learning rate increases gradually for the first 5 steps to stabilize early training.

Optimizer = AdamW 8-bit
→ Memory-efficient variant of the AdamW optimizer.

Max sequence length = 2048 tokens
→ Maximum length of input + output combined.

Weight decay = 0.01
→ Regularization to reduce overfitting.

Precisionfp = 16
→ The T4 GPU on Colab does not support bf16, so float16 is used instead.

Code Reference:

```
from trl import SFTTrainer                          # Import the Supervised Fine-Tuning Trainer from TRL
from transformers import TrainingArguments          # Import the training configuration class

trainer = SFTTrainer(                               # Create a fine-tuning trainer object
    model = model,                                  # The LoRA-wrapped model to train
    tokenizer = tokenizer,                          # The tokenizer to use for encoding the text
    train_dataset = dataset,                        # The formatted dataset to train on
    dataset_text_field = "text",                    # Name of the field in the dataset that contains the text to train on
    max_seq_length = max_seq_length,                # Maximum sequence length (2048 tokens, set earlier)
    dataset_num_proc = 2,                           # Number of parallel processes for dataset preprocessing
    args = TrainingArguments(                       # Define all training hyperparameters
        per_device_train_batch_size = 2,            # Process 2 examples per GPU per step
        gradient_accumulation_steps = 4,            # Accumulate gradients over 4 steps before updating weights; effective batch size = 2×4 = 8
        warmup_steps = 5,                           # Gradually increase the learning rate for the first 5 steps to stabilize early training
        num_train_epochs = 3,                       # Train for 3 full passes through the dataset
        learning_rate = 2e-4,                       # Step size for weight updates (0.0002)
        fp16 = not torch.cuda.is_bf16_supported(),  # Use float16 precision if bfloat16 is NOT supported by the GPU
        bf16 = torch.cuda.is_bf16_supported(),      # Use bfloat16 precision if the GPU supports it (T4 does not, so fp16 is used)
        logging_steps = 1,                          # Log the training loss after every single step
        optim = "adamw_8bit",                       # Use 8-bit AdamW optimizer — memory-efficient version of the standard optimizer
        weight_decay = 0.01,                        # L2 regularization to reduce overfitting
        lr_scheduler_type = "linear",               # Linearly decrease the learning rate from its peak to zero over training
        output_dir = "outputs",                     # Directory where model checkpoints are saved
        report_to = "none",                         # Disable reporting to external loggers like Weights & Biases or TensorBoard
    ),
)
```

Training Data Format:

Training examples were formatted using the **Alpaca prompt template**:

```
### Instruction:
Du bist ein Experte für österreichisches Steuerrecht. Beantworte die
folgende Frage präzise mit Angabe der relevanten Gesetzesparagraphen.

### Input:
[question]

### Response:
[reference answer]<EOS>
```

The *### Instruction* block defines the model's role, *### Input* contains the question (prompt) and *### Response* is what the model learns to generate. The end-of-sequence token *<EOS>* tells the model where to stop.

Inference Parameters:

These were the parameters used for generating answers on the test set.
max_new_tokens = 300
→ Generates up to 300 new tokens.

temperature = 0.1
→ Low temperature makes the output near-deterministic, meaning system will produce almost the same output for the same input (less random and probabilistic), with only minimal variation, which is important for a use case like law.

do_sample = True
→ The model generates output by sampling from possible next tokens, introducing randomness instead of always choosing the most likely option.

Code Reference:

```
# Formating promt for the question
prompt = alpaca_prompt.format(                    # Build the Alpaca-formatted prompt for this question
    "Du bist ein Experte für österreichisches Steuerrecht. Beantworte die folgende Frage präzise mit Angabe der relevanten Ge
    row["prompt"],                                # The question from this row of the CSV
    ""                                            # Empty response slot — to be generated by the model
)

# Tokenizing and onto GPU
inputs = tokenizer(                               # Tokenize the formatted prompt
    prompt,
    return_tensors="pt",                          # Return PyTorch tensors
    truncation=True,                              # Truncate if the prompt is too long
    max_length=1800                               # Limit prompt to 1800 tokens to leave room for the 300-token response
).to("cuda")                                      # Move the tokenized input to the GPU

# Generating answers
with torch.no_grad():                             # Disable gradient computation — not needed during inference, saves memory
    output = model.generate(                      # Generate the model's response
        **inputs,                                 # Pass tokenized input
        max_new_tokens = 300,                     # Generate up to 300 new tokens
        use_cache = True,                         # Enable KV-cache for faster decoding
        pad_token_id = tokenizer.eos_token_id,    # Use EOS token as padding to suppress warnings
        temperature = 0.1,                        # Low temperature = more deterministic, less random output
        do_sample = True,                         # Enable sampling (required when temperature is set)
    )

# Only decoding the generated text (without the promt)
answer = tokenizer.decode(                        # Decode generated tokens back to text
    output[0][inputs["input_ids"].shape[1]:],     # Skip the prompt tokens, only decode the new ones
    skip_special_tokens=True                      # Remove special tokens from the output
).strip()                                         # Remove leading/trailing whitespace

results.append({                                  # Add this result to the list
    "id": row["id"],                              # Include the question's ID for identification
    "answer": answer                              # Include the model's generated answer
```

<u>Platform used:</u>

Google Colab, T4 GPU (16 GB VRAM).

2.3 Qwen 2.5 3B + multilingual-E5-base – RAG

For the third and last model, we had to build a Retrieval-Augmented Generation (RAG) pipeline, which retrieves the most relevant chunks from actual Austrian law texts and then provides those chunks to the model as context before asking it to answer the question.
The pipeline consists of two components: The retriever (finds the relevant law text) and the generator (produces the answer given the retrieved text).

Retriever: intfloat/multilingual-e5-base

The retriever converts both the question and every law text chunk into a dense vector (a list of numbers representing the semantic meaning of the text). At query time, the vectors of all chunks are compared to the question vector, and the most similar chunks are returned.

Retriever Model Details:

→ Model: intfloat/multilingual-e5-base

→ Vector dimension: 768

→ Pre-training: Multilingual contrastive learning across 100+ languages

→ Index type: FAISS IndexFlatIP (exact cosine similarity via normalized inner product)

→ top_k: 5 chunks retrieved per question

→ Query prefix: “query: ” prepended to questions (required by the E5 model to distinguish queries from passages)

Code Reference:

```
from sentence_transformers import SentenceTransformer  # Library for loading pre-trained embedding models
import faiss                                           # Facebook's vector similarity search library

print("Lade Retriever...")                        # Announce model download/loading
retriever = SentenceTransformer("intfloat/multilingual-e5-base")
                                                  # Load the multilingual E5 embedding model — handles German text well
print("Retriever geladen")                        # Confirm the model is ready

print("\nWandle Chunks in Vektoren um...")         # Announce the embedding step
chunk_texte = [c['text'] for c in chunks]         # Extract just the text strings from the chunk dicts into a plain list

chunk_vektoren = retriever.encode(                # Convert all chunk texts into dense vector representations
    chunk_texte,
    show_progress_bar=True,                       # Show a progress bar since this takes a while with 6685 chunks
    batch_size=64,                                # Process 64 chunks at a time for memory efficiency
    convert_to_numpy=True                         # Return numpy arrays (required by FAISS)
)

print(f"Vektoren erstellt: {chunk_vektoren.shape}")  # Print the shape, e.g. (6685, 768) — 6685 chunks, 768 dimensions each

dimension = chunk_vektoren.shape[1]               # Extract the vector dimension (768 for E5-base)
faiss.normalize_L2(chunk_vektoren)                # Normalize all vectors to unit length — required for cosine similarity via inner product

index = faiss.IndexFlatIP(dimension)              # Create a FAISS flat index using Inner Product (= cosine similarity after normalization)
index.add(chunk_vektoren)                         # Add all chunk vectors to the index

print(f"FAISS Index aufgebaut mit {index.ntotal} Vektoren")  # Confirm how many vectors are in the searchable index
```

Generator: Qwen/Qwen2.5-3B-Instruct

For the generator I chose Qwen 2.5, as it performs well on German-language tasks despite its small size (3B parameters). Another reason for this decision was that it fits within Kaggle GPU memory limits after quantization.

Generator Model Details:

→ Model: Qwen/Qwen2.5-3B-Instruct

→ Parameters: 3 billion

→ Architecture: Decoder-only transformer (Alibaba Cloud)

→ Pre-training data: around 18 trillion tokens of multilingual text and code

→ Quantization: 4-bit NF4 (BitsAndBytes), double quantization, float16 compute

Code Reference:

```python
from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig
                                                    # AutoTokenizer: loads the tokenizer matching the model
                                                    # AutoModelForCausalLM: loads the text-generation model
                                                    # BitsAndBytesConfig: configures 4-bit quantization

model_name = "Qwen/Qwen2.5-3B-Instruct"            # Use Alibaba's Qwen 2.5 3B instruction-tuned model

quantization_config = BitsAndBytesConfig(           # Define 4-bit quantization settings to reduce VRAM usage
    load_in_4bit=True,                              # Load model weights in 4-bit precision
    bnb_4bit_quant_type="nf4",                      # Use NormalFloat4 quantization type — better quality than plain int4
    bnb_4bit_compute_dtype=torch.float16,           # Perform actual computations in float16 for speed
    bnb_4bit_use_double_quant=True                  # Apply double quantization to further compress the quantization constants
)

print("Lade Tokenizer...")                          # Announce tokenizer loading
tokenizer = AutoTokenizer.from_pretrained(model_name)  # Download and load the tokenizer for Qwen 2.5

print("Lade Modell auf GPU...")                     # Announce model loading
model = AutoModelForCausalLM.from_pretrained(       # Download and load the model weights
    model_name,
    quantization_config=quantization_config,        # Apply the 4-bit quantization defined above
    device_map="auto",                              # Automatically distribute the model across available GPUs
    trust_remote_code=True                          # Allow Qwen's custom model code to run (required by this model)
)
model.eval()                                        # Switch model to evaluation mode — disables dropout and gradient tracking
```

<u>Document collection and preprocessing:</u>

To assemble the relevant document collection, I downloaded Austrian law texts from the RIS API and pre-processed them in four steps *(see notebooks windisch_rag_doc-collection_1 through _4)*:

1. **Download**: Retrieving four laws via RIS API: EStG (law no. 10004570), UStG (10004873), KStG (10005025), GrEStG (10002382). Saving the raw JSON responses to the disk.
2. **Structure analysis**: Inspecting the response format to identify how paragraphs are organised within the JSON.
3. **Text extraction**: Cleaning each paragraph's raw text by stripping the metadata header (Kurztitel, BGBl references, Kundmachungsorgan etc.) that RIS prepends to every entry.
4. **Chunking**: two-level splitting strategy:
    - **Level 1:** Splitting at numbered subsection markers (1), (2), (3) etc. (Same way how Austrian legal paragraphs are structured).
    - **Level 2:** If a subsection still exceeded 800 characters, it was further split at sentence boundaries
    - Discarding chunks shorter than 50 characters as non-informative.

Chunk statistics:

Law	Chunks
EStG 1988	4,280
UStG 1994	1,188
KStG 1988	897
GrEStG 1987	320
Total	**6,685**
Average chunk length	587 characters
Longest chunk	1,892 characters
Shortest chunk	56 characters

Each chunk is stored with metadata: *law name*, *paragraph number*, *chunk number*, and a *source citation string* (e.g., "EStG 1988 § 23").

Code Reference:

```
def retrieve_chunks(frage, top_k=5):                # Function: find the most relevant law chunks for a given question
    frage_vektor = retriever.encode(                # Embed the question into a vector
        ["query: " + frage],                        # Prepend "query: " — required by the E5 model for query-mode encoding
        convert_to_numpy=True                       # Return as numpy array for FAISS
    )
    faiss.normalize_L2(frage_vektor)                # Normalize the query vector to unit length (same as the chunk vectors)
    scores, indices = index.search(frage_vektor, top_k)  # Search the FAISS index for the top_k most similar chunks
    return [chunks[idx] for idx in indices[0] if idx != -1]  # Return the matching chunk dicts; skip -1 (no match found)
```

<u>RAG prompt structure:</u>

For each question, the 5 retrieved chunks were formatted as:

```
[EStG 1988 § 23]
<chunk text>

[KStG 1988 § 8]
<chunk text>
...

Frage: <question>
```

The system prompt instructed the model to answer exclusively in German and exclusively on the basis of the provided law texts, citing the relevant paragraph.

Code Reference:

```python
def generate_answer(frage, relevante_chunks):      # Function: generate a German answer using the retrieved chunks as context
    kontext_teile = []                             # List to build the context string piece by piece
    for chunk in relevante_chunks:                 # Loop through each retrieved chunk
        kontext_teile.append(f"[{chunk['quelle']}]\n{chunk['text']}")  # Format each chunk as "[EStG 1988 § 23]\n<text>"
    kontext = "\n\n".join(kontext_teile)           # Join all formatted chunks into one context block

    messages = [                                   # Build the chat message list in the format Qwen expects
        {
            "role": "system",                      # System message: defines the model's role and constraints
            "content": (
                "Du bist ein Experte für österreichisches Steuerrecht. "  # Role definition
                "Beantworte die Frage ausschließlich auf Deutsch und ausschließlich "  # Answer must be in German only
                "auf Basis der bereitgestellten Gesetzestexte. "          # Answer must be based solely on the provided law texts
                "Antworte präzise und nenne den relevanten Paragraphen."  # Be concise and cite the relevant paragraph
            )
        },
        {
            "role": "user",                        # User message: contains the retrieved context and the question
            "content": f"Gesetzliche Grundlagen:\n{kontext}\n\nFrage: {frage}"
                                                   # Structure: law text chunks first, then the actual question
        }
    ]

    # apply_chat_template nur zum Text generieren, dann separat tokenisieren
    text = tokenizer.apply_chat_template(          # Convert the messages list into a single formatted string
        messages,
        tokenize=False,                            # Return a plain string, not token IDs yet
        add_generation_prompt=True                 # Append the assistant turn start marker so the model knows to generate
    )
```

Platform:

Kaggle, T4 GPU (16 GB VRAM).

3 Code

In this report I included code snippets occasionally, where they fit naturally within the paragraph. The remaining details can be looked up: All model notebooks, as well as the evaluation script, are available in the GitHub repository.
The relevant files are:

File	Description
windisch_inference_code.ipynb	GPT-2 Large inference pipeline
windisch_sft_code.ipynb	Llama 3.1 8B QLoRA fine-tuned
windisch_rag_doc-collection_1prepare-docs.ipynb	RIS API download
windisch_rag_doc-collection_2check-structure.ipynb	JSON structure analysis
windisch_rag_doc-collection_3downloading.ipynb	Full paragraph download
windisch_rag_doc-collection_4chunks.ipynb	Text cleaning and chunking
windisch_rag_code.ipynb	RAG pipeline
windisch_evaluation_code.ipynb	ROUGE and BERTScore evaluation

4 Model Evaluation

4.1 Metrics

For evaluating the performance of all three of my models against the reference answers from the common course dataset, I used four automated metrics from the rouge-score and bert-score libraries:

ROUGE-1 F1:

This metric measures the overlap of individual words (unigrams) between the generated answer and the reference answer. It computes precision (what fraction of the model's words appear in the reference), recall (what fraction of the reference words appear in the model's output), and combines them into an F1 score. Stemming was applied (*use_stemmer* = *True*), so forms like "steuerlichen" and "steuerlich" are treated as the same word.

ROUGE-2 F1:

This metric goes a step further by measuring the overlap of consecutive word pairs (bigrams). This is a stricter metric than ROUGE-1, as it rewards answers that not only use the right words but also arrange them in the same order as the reference. For short, precise legal answers this metric is particularly low across all models, because even a correct answer phrased slightly differently will share few bigrams with the reference.

ROUGE-L F1:

This metric measures the longest common subsequence (LCS) between the generated answer and the reference. Unlike ROUGE-2, it does not require the matching words to be neighbouring, as it captures sentence-level structural similarity. It falls between ROUGE-1 and ROUGE-2 in strictness.

BERTScore F1:

Whereas the rouge-scores measure "surface level" word overlap, the bert-score looks at semantic similarities. It embeds both the generated answer and the reference into a vector space using distilbert-base-multilingual-cased and computes similarity between token representations. Two answers can score well even if they use different words, as long as they have the same meaning. Values for in-domain German text typically range from roughly 0.75 (weak) to 0.90+ (strong).

All four metrics were computed pairwise for all 643 questions.

4.2 Results

Main result table:

```
=== Results Table ===
                          Model  ROUGE-1 F1  ROUGE-2 F1  ROUGE-L F1  BERTScore F1
    GPT-2 large (Inference only)      0.0821      0.0108      0.0600        0.7743
Llama 3.1 8B (Fine-Tuned, QLoRA)      0.2427      0.0736      0.1654        0.8256
 Qwen2.5 + multilingual-E5 (RAG)      0.2352      0.0755      0.1593        0.8129
```

The **fine-tuned Llama 3.1 8B model performs best** on **all four metrics**.
The RAG system is close behind on all ROUGE variants and BERTScore, despite having fewer parameters and no domain-specific fine-tuning. GPT-2 Large fails at this task almost entirely across all metrics.

The very low ROUGE-2 scores across all models (0.01-0.08) reflect how strict bigram matching is for this task. Reference answers are short and precisely worded and even a semantically correct answer phrased differently will share almost no consecutive word pairs with the reference. ROUGE-L values fall between ROUGE-1 and ROUGE-2 as expected. BERTScore is the most forgiving metric and likely the most informative for this task, as it captures whether the answer conveys the correct legal concept, even when the exact wording differs.

Score distribution across all 643 samples given BERTScore:

Model	Samples BERTScore < 0.80	Samples BERTScore > 0.85
GPT-2 Large	536 / 643 **(83%)**	4 / 643 (1%)
Llama 3.1 8B SFT	123 / 643 (19%)	131 / 643 **(20%)**
Qwen 2.5 3B RAG	233 / 643 (36%)	75 / 643 (12%)

One consistent limitation across all models is a verbosity mismatch. While reference answers are very brief, often only one sentence with a paragraph citation, my three models tend to produce longer explanations. This suppresses rouge-scores, even if the answers are correct, meaning the raw numbers probably understate the true answer quality for the fine-tuned and RAG models.

4.3 Error Analysis

Per-sample scores were used to identify the worst-performing examples using ROUGE-1 F1 and BERTScore and examine systematic failure patterns per model.

GPT-2 Large (Inference Only)

GPT-2 was pre-trained exclusively on English web text from Reddit. It was never instruction-tuned and has no knowledge of German or Austrian tax law. It fails almost completely at this task.

Main failure modes:

1. Wrong language: The model generates English text in response to German questions. It does not understand that the prompt is in German or that a German answer is expected.

Example prompt:
“Was ist ein Reverse Stock Split steuerlich?”

GPT-2 output:
“If you can't get the answer to the above questions, then please contact us...”

2. Hallucinated web content: The model continues the prompt as if completing a web page where it pastes forum posts, patent filing text, and other irrelevant material from its training data.

Example: Question about cryptocurrency taxation.

GPT-2 output:
“Gavin Andresen Offline Activity: 1652 Merit: 1014 Chief Scientist...” (This is from a Bitcoin forum post that the model has memorised).

3. No legal reasoning: The model cannot cite Austrian law paragraphs because it has never seen them.

Side by side comparison of the faulty answers:

ID	**Reference** (truncated)	**GPT-2 Output** (truncated)	**BERTScore**
ESTG-NEW-019	*Der Reverse Stock Split an sich ist für die glatten Anteile kein steuerpflichtiger Vorgang...*	*If you can't get the answer to the above questions, then please contact us...*	0.697

ESTG-NEW-012	*Nein. Kryptowährungen, die zum Betriebsvermögen gehören, fallen nicht unter § 27b EStG...*	*Gavin Andresen Offline Activity: 1652 Merit: 1014 Chief Scientist...*	0.698
TAX-INTL-075	*Ja. Der Verlustabzug steht beschränkt Steuerpflichtigen nach § 102 Abs. 2 Z 2 EStG 1988…*	*Ce n'est pas plus de plus de plus de plus de plus de plus de plus de…*	0.707
VAT-INTL-001	*Es handelt sich um eine kurzfristige Vermietung eines Beförderungsmittels an eine Privatperson (B2C)…*	*[…] "Was it a matter of one's own choice whether to be a Jew or a Christian?"…*	0.707

Llama 3.1 8B (Fine-Tuned)

The fine-tuned model generates fluent, on-topic German legal text and correctly cites paragraphs in most cases. Its failures are far more subtle.

Main failure modes:

1. Verbosity mismatch: Reference answers are often a single sentence (e.g., *"Als Bemessungsgrundlage für die Grundsteuer und die landwirtschaftliche Sozialversicherung."*). The model consistently produces multi-sentence explanations. While the content may be correct, ROUGE-1 is lower because the model's extra words don't appear in the short reference.

2. Wrong paragraph citation: The model occasionally identifies the correct legal concept but cites the wrong paragraph — citing § 10 Abs. 1 Z 2 EStG where the reference answer expects § 7 Abs. 1 Z 1 lit. b GrEStG. Additionally, contradicting the yes/no conclusions.

3. Hallucinated legislative history: For questions about recent legislative changes, the model generates plausible sounding but fabricated details which are not actually present in the reference.

Side by side comparison of the faulty answers:

ID	**Reference** (truncated)	**Llama output** (truncated)	**BERTScore**
SELF-018	Als Bemessungsgrundlage für die Grundsteuer und die landwirtschaftliche Sozialversicherung.	Der Einheitswert (EV) ist eine Grundlage für verschiedene Abgaben: (1) GrESt: Der Grundstückswert...	0.741
SELF-025	Anpassungen bei den Steuersätzen und Befreiungen für Betriebsübergaben.	Das Budgetbegleitgesetz 2001 (BGBl. I 2001/109) führte folgende Änderungen...	0.744

Qwen 2.5 3B + RAG

The RAG model's quality depends entirely on whether the retrieval step collects the correct chunks. When the retrieval succeeds, the answer is based on an actual law text. When it fails, the model either refuses to answer or starts making up information.

Main failure modes:

1. Retrieval failure / honest refusal: The model explicitly states it cannot find the answer in the provided context. This is semantically correct behaviour, but it naturally scores very low against referenced answers.

Example output:
“Basierend auf den bereitgestellten Gesetzestexten kann ich leider keine genauen vorgeschriebenen Voraussetzungen finden...” This typically happens when the question targets a very specific provision and the retrieved chunks come from the wrong subsection.

2. Wrong law retrieved: Some questions span multiple laws. A question about GrEStG might retrieve EStG chunks because the embedding model finds lexical similarity (e.g. both mentioning “Grundstück”) without distinguishing which law the answer must come from.

3. Verbosity mismatch: This is the same issue as with Llama, having short reference answers score poorly against longer grounded explanations.

Side by side comparison of the faulty answers:

ID	Reference (truncated)	RAG output (truncated)	BERTScore
GRESt-AT-026	Gemäß § 3 Abs. 1 Z 2 lit. b GrEStG muss der Übergeber das 55. Lebensjahr...	Basierend auf den bereitgestellten Gesetzestexten kann ich leider keine genauen...	0.733
SELF-042	Durch Betriebsvermögensvergleich (Bilanzierung).	Der Gewinn nach § 4 Abs. 1 EStG wird durch die Differenz zwischen dem Erlösen...	0.740

5 Conclusion

The **fine-tuned Llama 3.1 8B** model achieves the best performance on both metrics (ROUGE-1: 0.2427, BERTScore: 0.8256), demonstrating that **domain-specific** fine-tuning with even a small synthetic training set of 253 examples significantly improves answer quality on Austrian tax law questions.

The RAG system based on Qwen 2.5 3B is a competitive alternative without any fine-tuning. Its ROUGE-1 score (0.2352) is very close to Llama's, and it benefits from access to actual law texts at inference time, which makes it less prone to fabrication. Its main weakness is retrieval quality, when the wrong chunks are returned, the model either refuses to answer or generates incorrect responses. Improvements such as a larger embedding model, a hybrid retrieval strategy, or law-specific chunk metadata filtering would likely close the remaining gap.

GPT-2 Large is not viable for this task. As it is pre-trained on English web text without instruction tuning, it has no knowledge of German or Austrian law and effectively produces random continuations of the input prompt. Its inclusion serves solely as a lower bound baseline.

A limitation shared by all three models is the verbosity mismatch between generated outputs and reference answers. Reference answers are very concise, while all models produce longer explanations. This structurally suppresses ROUGE scores even for correct answers, meaning absolute metric values likely understate true answer quality for the fine-tuned and RAG models.

6 Sources

Models:

- GPT-2 Large: Radford et al. (2019). *Language Models are Unsupervised Multitask Learners.* OpenAI. https://huggingface.co/gpt2-large
- Llama 3.1: Meta AI (2024). *Llama 3.1 Model Card.* https://huggingface.co/meta-llama/Meta-Llama-3.1-8B-Instruct
- Qwen 2.5: Alibaba Cloud (2024). *Qwen2.5 Technical Report.* https://huggingface.co/Qwen/Qwen2.5-3B-Instruct
- multilingual-E5: Wang et al. (2024). *Multilingual E5 Text Embeddings.* https://huggingface.co/intfloat/multilingual-e5-base

Libraries:

- Unsloth: https://github.com/unslothai/unsloth
- TRL (SFTTrainer): https://github.com/huggingface/trl
- FAISS: Johnson et al. (2019). *Billion-scale similarity search with GPUs.* https://github.com/facebookresearch/faiss
- rouge-score: https://pypi.org/project/rouge-score/
- bert-score: Zhang et al. (2020). *BERTScore: Evaluating Text Generation with BERT.* https://github.com/Tiiiger/bert_score

Data:

- Austrian Legal Information System (RIS): https://data.bka.gv.at/ris/api/v2.6/Bundesrecht
- Course dataset: Vakulenko, S. (2026). *wu-llms-ss26 dataset_clean.csv.* https://github.com/svakulenk0/wu-llms-ss26

Background:

- Hu et al. (2022). *LoRA: Low-Rank Adaptation of Large Language Models.* ICLR 2022. https://arxiv.org/abs/2106.09685
- Dettmers et al. (2023). *QLoRA: Efficient Finetuning of Quantized LLMs.* NeurIPS 2023. https://arxiv.org/abs/2305.14314
- Lewis et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* NeurIPS 2020. https://arxiv.org/abs/2005.11401
- Evaluation metrics overview: https://medium.com/@pur4v/understanding-llm-evaluation-metrics-bleu-rouge-exact-match-and-bertscore-716487e40bdd